

# **Hitec University Taxila, Department of Computer Engineering**

## **Lab Manual for Mobile Application Development**

Lab-10 SQLite Database

# Contents

|   |                                   |    |
|---|-----------------------------------|----|
| 1 | Introduction .....                | 3  |
| 2 | Activity Time boxing .....        | 3  |
| 3 | Objectives of the experiment..... | 4  |
| 4 | Walk through Task .....           | 4  |
| 5 | Home Task .....                   | 10 |

## 1 Introduction

Structure Query Language(SQL) is a database query language used for storing and managing data in Relational Database Management System (DBMS). The APIs you'll need to use a database on Android are available in the “android.database.sqlite”**Adapter”**

### Contract Class in SQLite

A contract class is a container for constants that define names for tables, and columns. The contract class allows you to use the same constants across all the other classes in the same package. This lets you change a column name in one place and have it propagate throughout your code. A good way to organize a contract class is to put definitions that are global to your whole database in the root level of the class.

### SQLite Helper Class

Once you have defined how your database looks, you should implement methods that create and maintain the database and tables using the “SQLiteOpenHelper” class. The “SQLiteOpenHelper” class contains a useful set of APIs for managing your database. The database can be managed by calling “getWritableDatabase()” or “getReadableDatabase()” callbacks in helper class. When you extend your helper class with “SQLiteOpenHelper” two callbacks must be implemented

- onCreate()
- onUpgrade()

### “ContentValues” for Entire SQLite

ContentValues is a map-like class that matches a value to a String key. It contains multiple overloaded put methods that enforce type safety. Here is a list of the put methods supported by ContentValues

- void put(String key, String value)
- void put(String key, Integer value)
- String getString(String Key)
- Integer getInt(String Key)
- Integer getColumnIndex(String ColumnName)

### Read All Entries from SQLite Table

The SQL query can be implemented to read all entries from table by executing the “rawQuery()” method, which will return the “Cursor”. Cursors are what contain the result set of a query made against a database in Android. The Cursor class has an API that allows an app to read (in a type-safe manner) the columns that were returned from the query as well as iterate over the rows of the result set

## 2 Activity Time boxing

Table 1: Activity Time Boxing

| Task# | Activity Name     | Activity time | Total Time |
|-------|-------------------|---------------|------------|
| 1     | Introduction      | 60mins        | 60min      |
| 2     | Walk Through Task | 60mins        | 60min      |
| 3     | Evaluation Task   | 60min         | 60min      |

### 3 Objectives of the experiment

- To get basic understanding regarding SQLite
- To get basic understanding regarding how SQLite use to design database in android studio
- To get to know about the benefits related to SQLite

### 4 Walk through Task

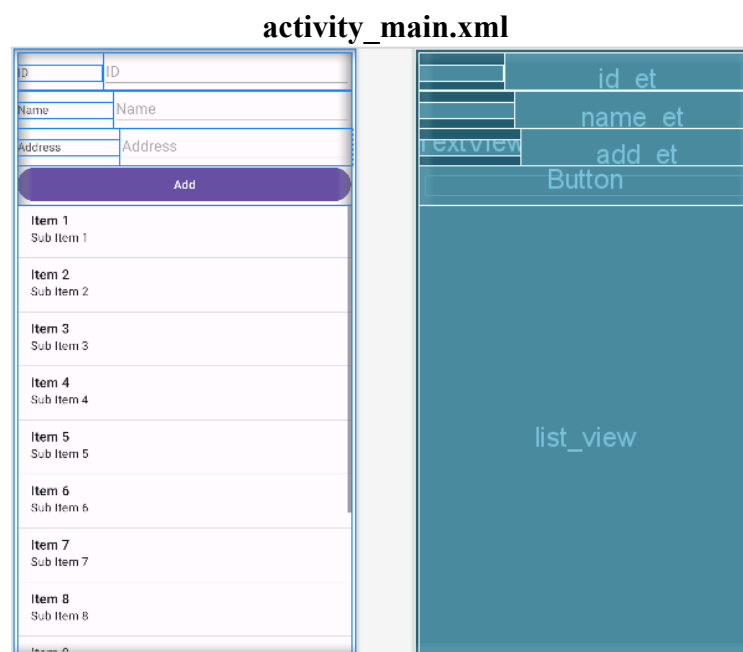
#### Creating New Activity

##### Step 1: Create Activities:

Create a new project just like you have created in your previous labs and create one more empty activity.

##### Step 2: Design XMLs of Activity:

Designs of activity are given below, just use the given xml code which is given in this step.



```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:padding="5dp"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal">

        <TextView
            android:id="@+id/id_name"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:text="ID" />
    </LinearLayout>
</LinearLayout>
```

```

<EditText
    android:id="@+id/id_et"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_weight="1"
    android:ems="10"
    android:inputType="textPersonName"
    android:hint="ID" />

</LinearLayout>

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal">

    <TextView
        android:id="@+id/name_tc"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:text="Name" />

    <EditText
        android:id="@+id/name_et"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:ems="10"
        android:inputType="textPersonName"
        android:hint="Name" />

</LinearLayout>

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal">

    <TextView
        android:id="@+id/add_tc"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:text="Address" />

    <EditText
        android:id="@+id/add_et"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:ems="10"
        android:inputType="textPersonName"
        android:hint="Address" />

</LinearLayout>

<LinearLayout
    android:layout_width="match_parent"

```

```

        android:layout_height="wrap_content"
        android:orientation="horizontal">

        <Button
            android:id="@+id/add_btn"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:onClick="addNewEntry"
            android:text="Add" />

    </LinearLayout>

    <ListView
        android:id="@+id/list_view"
        android:layout_width="match_parent"
        android:layout_height="wrap_content" />
</LinearLayout>

```

### Step 3: Write “SchoolContract.java” contract class

```

public final class SchoolContract {

    public static class Student {
        // Tables and Column names
        public static final String TABLE_NAME = "student";
        public static final String COLUMN_ID = "id";
        public static final String COLUMN_NAME = "name";
        public static final String COLUMN_ADDRESS = "address";

        // SQL queries
        public static final String CREATE_STUDENT_TABLE =
            "CREATE TABLE " + TABLE_NAME + "(" +
                COLUMN_ID + " INTEGER PRIMARY KEY, " +
                COLUMN_NAME + " TEXT, " +
                COLUMN_ADDRESS + " TEXT) ";

        public static final String DELETE_STUDENT_TABLE =
            "DROP TABLE IF EXISTS " + TABLE_NAME;
        public static final String READ_ALL_TABLE = "SELECT * FROM " +
            TABLE_NAME;
    }
}

```

### Step 4: Write “SchoolDBHelper.java” SQLite DB helper class

```

SchoolDBHelper.java

import android.content.ContentValues;
import android.content.Context;
import android.database.Cursor;
import android.database.sqlite.SQLiteDatabase;
import android.database.sqlite.SQLiteOpenHelper;
import android.widget.Toast;

import androidx.annotation.Nullable;

```

```

import java.util.ArrayList;

public class SchoolDBHelper extends SQLiteOpenHelper {
    Context context;
    public static final int DATABASE_VERSION = 1;
    public static final String DATABASE_NAME = "School.db";

    public SchoolDBHelper(@Nullable Context context) {
        super(context, DATABASE_NAME, null, DATABASE_VERSION);
        this.context = context;
    }

    @Override
    public void onCreate(SQLiteDatabase db) {
        db.execSQL(SchoolContract.Student.CREATE_STUDENT_TABLE);
    }

    @Override
    public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
        // This database is only a cache for online data, so its upgrade policy
is
        // to simply to discard the data and start over
        db.execSQL(SchoolContract.Student.DELETE_STUDENT_TABLE);
        onCreate(db);
    }
    // add id, name, address to table
    public void addEntry(int id, String name, String address){
        SQLiteDatabase db = getWritableDatabase();
        // Create a new map of values, where column names are the keys
        ContentValues values = new ContentValues();
        values.put(SchoolContract.Student.COLUMN_ID, id);
        values.put(SchoolContract.Student.COLUMN_NAME, name);
        values.put(SchoolContract.Student.COLUMN_ADDRESS, address);
        try{
            // Insert the new row
            db.insertOrThrow (SchoolContract.Student.TABLE_NAME, null, values);
        }catch (Exception e){
            Toast.makeText(context, "Error: " + e, Toast.LENGTH_SHORT).show();
        }
        // must close the db
        db.close();
    }
    public ArrayList<SchoolSetter> readAll() {
        SQLiteDatabase db = getWritableDatabase();
        Cursor cursor = db.rawQuery(SchoolContract.Student.READ_ALL_TABLE,null);

        ArrayList<SchoolSetter> setters = new ArrayList<SchoolSetter>();
        while (cursor.moveToNext()) {
            setters.add(new SchoolSetter(
                cursor.getInt(0),
                cursor.getString(1),
                cursor.getString(2)
            ));
        }
        return setters;
    }
}

```

### Step 5: Write “SchoolSetter.java” class

#### SchoolSetter.java

```
public class SchoolSetter {
    int id;
    String name;
    String address;

    public SchoolSetter(int id, String name, String address) {
        this.id = id;
        this.name = name;
        this.address = address;
    }

    public int getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    public String getAddress() {
        return address;
    }
}
```

### Step 6: Write “SchoolBaseAdapter.java” class

#### SchoolBaseAdapter.java

```
import android.content.Context;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.BaseAdapter;
import android.widget.TextView;

import java.util.ArrayList;

public class SchoolBaseAdapter extends BaseAdapter {
    ArrayList<SchoolSetter> setter;
    LayoutInflater inflater;
    public SchoolBaseAdapter(Context context, ArrayList<SchoolSetter> setter) {
        this.setter = setter;
        inflater = LayoutInflater.from(context);
    }

    @Override
    public int getCount() {
        return setter.size();
    }

    @Override
    public Object getItem(int position) {
        return setter.get(position);
    }

    @Override
```

```

public long getItemId(int position) {
    return position;
}

@Override
public View getView(int position, View convertView, ViewGroup parent) {
    // get view of custom item layout
    convertView = inflater.inflate(R.layout.list_item_sql, null);
    TextView id = convertView.findViewById(R.id.item_id);
    TextView name = convertView.findViewById(R.id.item_name);
    TextView address = convertView.findViewById(R.id.item_address);

    id.setText(String.valueOf(setter.get(position).getId()));
    name.setText(String.valueOf(setter.get(position).getName()));
    address.setText(String.valueOf(setter.get(position).getAddress()));
    return convertView;
}
}

```

## Step 6: Write “MainSQLiteActivity.java” class

```

import androidx.appcompat.app.AppCompatActivity;

import android.os.Bundle;
import android.view.View;
import android.widget.EditText;
import android.widget.ListView;
import android.widget.Toast;

import java.util.ArrayList;

public class MainSQLiteActivity extends AppCompatActivity {
    EditText idET, nameET, addressET;
    SchoolDBHelper dbHelper;
    ListView listView;
    SchoolBaseAdapter adapter;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main_sqlite);

        idET = findViewById(R.id.id_et);
        nameET = findViewById(R.id.name_et);
        addressET = findViewById(R.id.add_et);

        listView = findViewById(R.id.list_view);
        // create DB
        dbHelper = new SchoolDBHelper(getApplicationContext());

        ArrayList<SchoolSetter> schoolSetters = dbHelper.readAll();
        adapter = new SchoolBaseAdapter(getApplicationContext(), schoolSetters);

        listView.setAdapter(adapter);
    }

    public void addNewEntry(View view) {
        if(!idET.getText().toString().matches("") ||
            !nameET.getText().toString().matches("") ||
            !addressET.getText().toString().matches("")) {

```

```

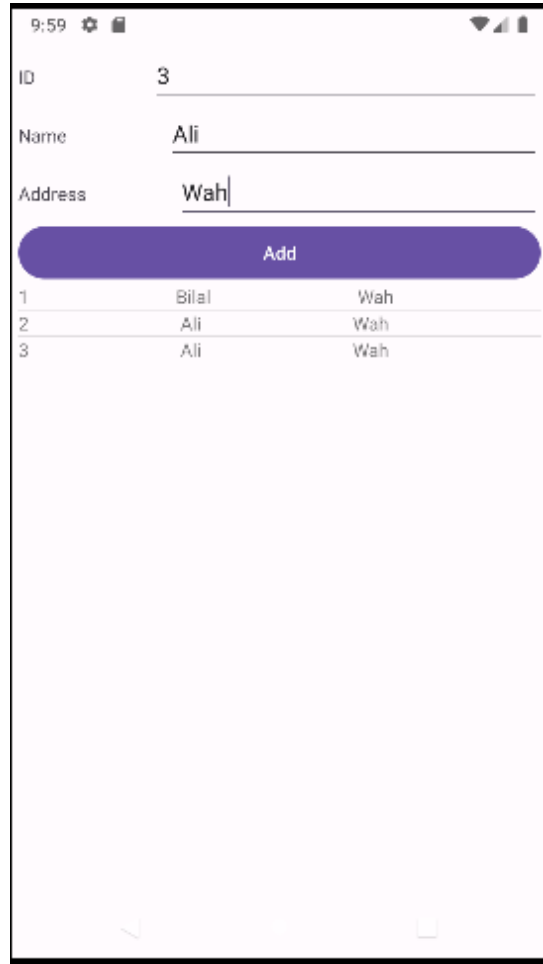
        int id = Integer.valueOf(idET.getText().toString());
        String name = nameET.getText().toString();
        String address = addressET.getText().toString();
        dbHelper.addEntry(id, name, address);
        updateListView();
    } else {
        Toast.makeText(this, "Fill all edit box",
            Toast.LENGTH_SHORT).show();
    }
}

void updateListView() {
    ArrayList<SchoolSetter> schoolSetters = dbHelper.readAll();
    adapter = new SchoolBaseAdapter(getApplicationContext(), schoolSetters);
    listView.setAdapter(adapter);
    listView.invalidateViews();
}
}

```

### Step 6: Now Run Project

Click "Run" or press "Shift + F10".



## 5 Home Task

Use Tasbeeh app and show history in view pager and store history in SQLite database.